

HPC@HRZ, UNI BONN

INTRODUCTION TO WORKING ON HPC CLUSTERS

Jan Steiner

March 11, 2026

<https://hpc.uni-bonn.de>



1. Theory part

- Cluster Structure
- HPC at Uni Bonn
- Job Scripts
- SLURM Parameters
- How does PyTorch do it?

2. Exercise Your First HPC Job

3. Dynamic part (might not have time for all)

- Linux environments
- File systems
- Interactive jobs

STRUCTURE OF A CLUSTER

- Computations can become too large for one computer
 - Too much data for RAM
 - Too much total data for hard drive
 - Execution time in months, years
 - Too many small problems (e.g. parameter study)

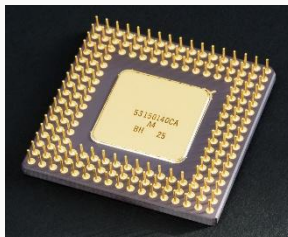
→ Use more computers

- High Performance Computing (HPC)
 - Cluster of computers
 - Individually: similar to (Linux) PC
 - But many, and interconnected
- Sometimes distinct: High-*Throughput* Computing → large numbers of small jobs

Central Processing Units (CPUs)

Smaller

- ~ 10-100 cores per CPU



- 1-4 CPUs per node



Larger

- A few - hundreds of nodes per cluster
 - In one or multiple racks



Other components

Node-level

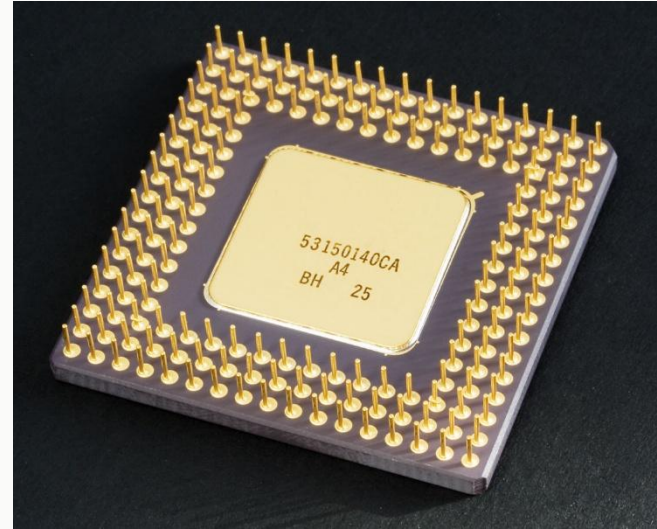
- RAM
- Sometimes: GPUs



Central

- Hard drives

- CPU: general purpose processor
- Multi-core: can do multiple things at once
 - Marvin: 48 cores per CPU
- Hyperthreading: usually disabled



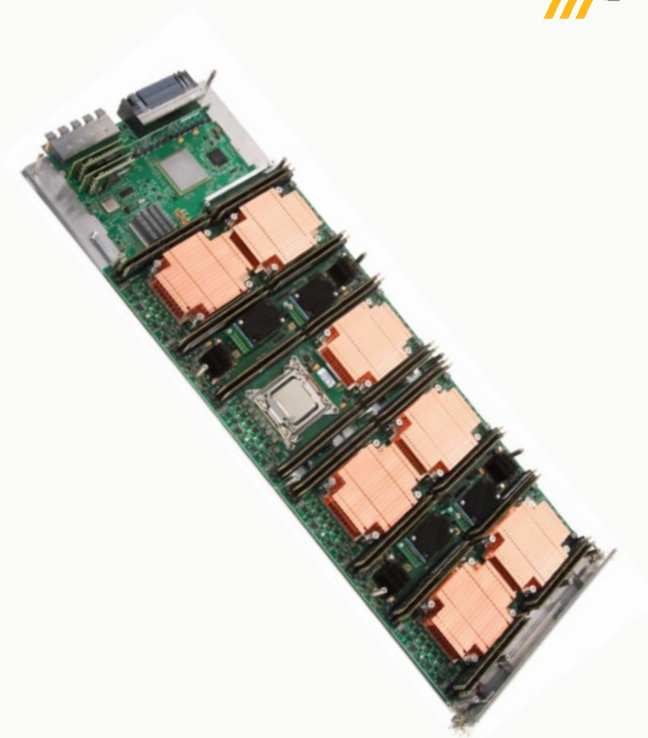
Source: Wikimedia Commons

- Graphics Processing Unit
 - Specialized processor
- Origin: 3D computer games
 - Same operation with large array of numbers (e.g. xyz coordinates)
- Science: similar workload



Source: Wikimedia Commons

- Node $\leftrightarrow$ (Linux) PC
- RAM: like PC, but more (Marvin: 1024 GB)
- Different types
 - By function: login, compute, management
 - By hardware: fat node == more RAM, with/without GPU



Source: Wikimedia Commons

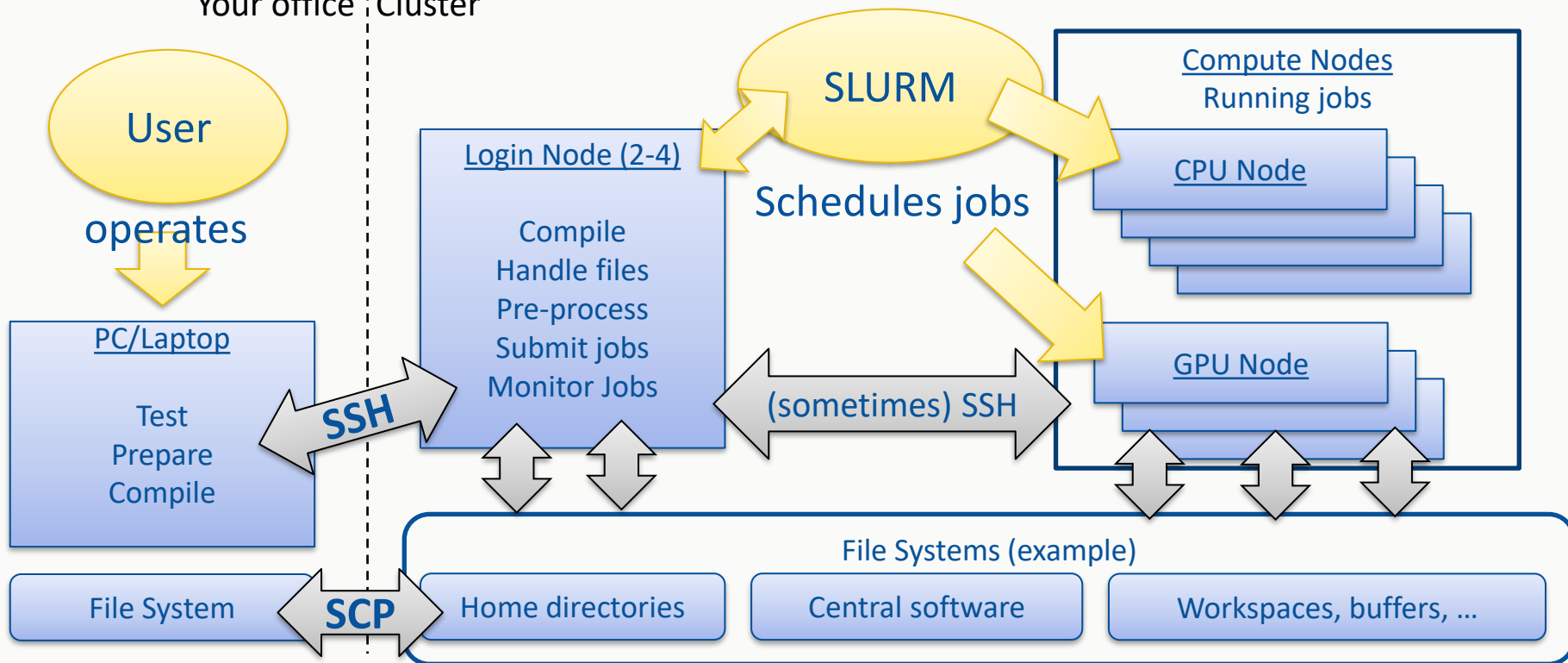
- Cooling
- Power supply
- Interconnect (network)
 - Faster than regular (Ethernet)
 - Makes cluster a cluster
 - Common: **Infiniband** (Bonna: Omnipath)



- One or more racks
 - Marvin: 15
- Usually in dedicated room
- Infrastructure (e.g. fire suppression)
- Central file storage
 - Sometimes nodes have own hard drives/SSDs



Your office | Cluster



Which of the following statements are true?

- *"A typical cluster contains a number of nodes, those contain several CPUs, those contain several cores"*
- *"In a cluster, each CPU core usually has its own RAM"*
- *"Computations are normally started by the user on the login node"*



<https://uni-bonn.sciebo.de/apps/forms/s/JHM3stdXsQfGn97f6xfj8LnM>

– *"A typical cluster contains a number of nodes, those contain several CPUs, those contain several cores"*

 – True

– *"In a cluster, each CPU core usually has its own RAM"*

 – False. Each **node** usually has its own RAM, which is shared between all its CPUs

– *"Computations are normally started by the user on the login node"*

 – False. The user puts the job into a queue, and the scheduling system runs them on the compute nodes

HPC AT UNI BONN



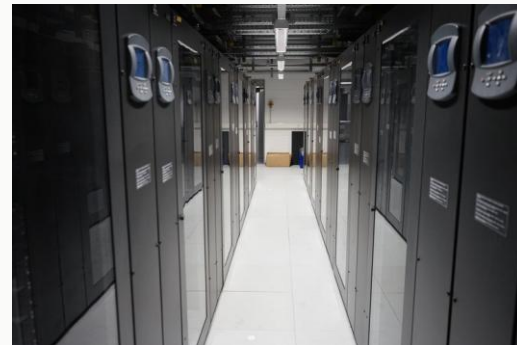
Bender

- Small
- GPUs
- For teaching (e.g. BSc theses)



Bonna

- Mid-size
- CPUs only
- For projects without need for GPUs



Marvin

- Large
- CPUs and GPUs
- Advanced projects
- Guest researchers

- Use systems responsibly!
 - You use limited resource
 - With relatively few technical restrictions
 - Easy to waste resources due to inexperience, laziness
 - We try to save you from yourself, but we are not perfect

→ High level of personal responsibility

- HPC Website: <https://www.hpc.uni-bonn.de>
 - Registration
 - Training courses: <https://www.hpc.uni-bonn.de/en/support/training>
- Marvin, Bender: own (small) wiki <https://wiki.hpc.uni-bonn.de/>
- General HPC topics: https://hpc-wiki.info/hpc/HPC_Wiki

1. What exactly were you doing?
 - Command/script/program
 - Job: attach SLURM job script to ticket

2. What exactly happened
 - Error message
 - Job: attach SLURM logfile to ticket

- Your SLURM job crashes. What two things should you always attach to the ticket?
- Which of the following is the correct support address for Bender?
 - bender@uni-bonn.de
 - bender@hpc.uni-bonn.de
 - bender-support@uni-bonn.de
 - bender-support@hpc.uni-bonn.de



<https://uni-bonn.sciebo.de/apps/forms/s/rgMy5EEkMwT28MXgPcyHxZTA>

- Your SLURM job crashes. What two things should you always attach to the ticket?
 - The SLURM job script that caused the error
 - The SLURM logfile that contains the error
- Which of the following is the correct support address for Bender?
 - bender-support@hpc.uni-bonn.de

JOB SCRIPTS

1. You write job script
2. You prepare software, input files, etc.
3. You queue your job script
4. You monitor you job when it runs

Job script

- Calls your software
- Provides job setting
- Load environment
- Any other necessary tasks

```
#!/bin/bash  
#SBATCH --gpus=1  
#SBATCH --time=0:20:00  
#SBATCH --partition=A40short  
#SBATCH --mem=100G  
#SBATCH --mail-user=mail@example.com  
#SBATCH --mail-type=ALL  
  
module load Python  
echo "This is job number $SLURM_JOB_ID"  
  
mkdir results  
python myscript.py
```

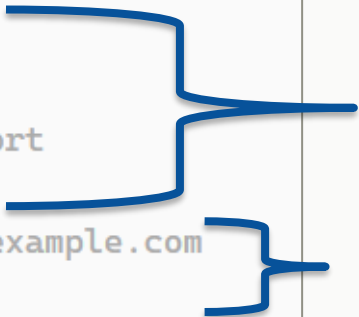
Which shell to use (Linux command)

- At least two different families (csh,bash)
- Different syntax
- Default on cluster: `bash`
- Does not have to be shell


```
#!/bin/bash
#SBATCH --gpus=1
#SBATCH --time=0:20:00
#SBATCH --partition=A40short
#SBATCH --mem=100G
#SBATCH --mail-user=mail@example.com
#SBATCH --mail-type=ALL

module load Python
echo "This is job number $SLURM_JOB_ID"

mkdir results
python myscript.py
```



SLURM settings

- Most important:
 - Which resources (GPUs, cores, RAM)
 - Which queue (partition)
 - For how long
- Additional settings
 - Defaults exist for most

```
#!/bin/bash
#SBATCH --gpus=1
#SBATCH --time=0:20:00
#SBATCH --partition=A40short
#SBATCH --mem=100G
#SBATCH --mail-user=mail@example.com
#SBATCH --mail-type=ALL

module load Python
echo "This is job number $SLURM_JOB_ID"

mkdir results
python myscript.py
```

Load environments

- As needed
- Env: Partially handed over from login shell
- Cleanest: define at beginning of job
- SLURM parameters for control

```
#!/bin/bash
#SBATCH --gpus=1
#SBATCH --time=0:20:00
#SBATCH --partition=A40short
#SBATCH --mem=100G
#SBATCH --mail-user=mail@example.com
#SBATCH --mail-type=ALL

module load Python
echo "This is job number $SLURM_JOB_ID"

mkdir results
python myscript.py
```

Additional tasks

- Prettier output
- File/folder setup as needed
- SLURM provides env vars about jobs

```
#!/bin/bash
#SBATCH --gpus=1
#SBATCH --time=0:20:00
#SBATCH --partition=A40short
#SBATCH --mem=100G
#SBATCH --mail-user=mail@example.com
#SBATCH --mail-type=ALL

module load Python
echo "This is job number $SLURM_JOB_ID"

mkdir results
python myscript.py
```

Call your program

- Might be arbitrarily complex
- Fmake sure it actually uses resources
- Reference SLURM env vars (how many cores etc.)

QUEUING A JOB SCRIPT WITH SBATCH

`sbatch` command
`man sbatch` for details

Options override script options / defaults

```
$ sbatch --partition=A40devel jobscript.sh
Submitted batch job 54428
```

SLURM will print job ID

Your job script

- Does not need to be executable
- But needs `#!/executable` at top
- E.g. `#!/bin/bash`

- Check regularly what job does
 - Your first job will fail (guaranteed)
 - Might be a bug later on
 - Might be a problem with cluster
 - Might run out of resources
 - Might not be finished when time limit reached
- Main command to check job: `squeue`
- *Checkpointing*: write intermediate results to hard drive

- `squeue`: overview what jobs are doing
 - Many view options:
 - Filter, i.e. only your jobs (`--me`)
 - Which fields
- `scancel <Job ID>`: abort job
 - `scancel --me`
- `scontrol shob job <Job ID>`: very detailed information (during run)

- Where does screen output go?
- SLURM creates logfile for every job
 - Default: `slurm-<Job ID>.out`
 - Into job directory (where you executed sbatch)
 - Error log: `slurm-<Job ID>.err`
- Filename, location etc. can be fine-tuned with `sbatch` options

SELECTING SLURM PARAMETERS

- Remember: your responsibility not to waste resources
- SLURM allows fine-tuning → Create model of your process structure
- Generally for every new job, think about:
 - How many processes, cores, memory
 - How long to set runtime
 - Which queue

```
--partition=short, -pshort
```

- Queues (SLURM: "partitions") may differ by
 - Runtime (short, medium, long)
 - Hardware (A40short, A100 short)
 - Purpose (devel)
- Usually:
 - Restrictions (max nodes, max runtime)
 - Defaults

```
--time=D-HH:MM:SS, --time=HH:MM:SS, -tM
```

- Request as little time as possible
 - Shorter wait times
- However: leave generous time reserve!
 - Time limit reached = hard kill
 - 50% reserve → not tragic
- Queue in shortest queue that it fits into

- How many CPU cores?
- Annoying SLURM terminology: process → task, core → CPU
- Many options:
 - `--ntasks=?` or `--nodes=?` + `--ntask-per-node=?`
 - `--cpus-per-task=?`
- Total number of cores: product of these numbers

- How much RAM?

- `--mem=500000,`

- `--mem=500G,`

- `--mem=0` everything on node

- Default:

- RAM proportionally to number of cores

- E.g. reserve 32 out of 96 cores on one node → third of RAM

- How to reserve GPUs
 - `--gpus=?`
 - Default: no GPU → software cannot even see GPUs
- Default:
 - CPU cores proportionally to number of GPUs
 - E.g. reserve 1 out of 4 GPUs on one node → quarter of cores
 - Also implies RAM proportion

- Marvin: your research group
 - `#SBATCH --account=ag_ukb_example`
- E-Mail notifications (not working on Bender)
- Modify environment
- Rename logfiles
- ...

Documentation: <https://slurm.schedmd.com/sbatch.html>

- `sinfo`: show (some) queue settings
 - Nodes, partitions listed multiple times
 - Asterisk (*): default queue
 - Status: free/occupied, up/down
 - Restrictions
- `scontrol show node, partition, ...`

- How many cores will SLURM allocate in each case?
- **Assume:** default number of tasks and CPUs per task are 1
- **Assume:** 32 cores and 4 GPUs per node
- **Assume:** cores proportionally allocated for each GPU (like Bender, Marvin)

1. `sbatch --nodes=2 --ntasks-per-node=16 quiz1.sh`

2. `sbatch --ntasks=48 --cpus-per-task=2 quiz2.sh`

3. `sbatch --gpus=1 quiz3`

4. `sbatch quiz4.sh`

- **Assume:** default number of tasks and CPUs per task are 1
- **Assume:** 32 cores and 4 GPUs per node
- **Assume:** cores proportionally allocated for each GPU (like Bender, Marvin)



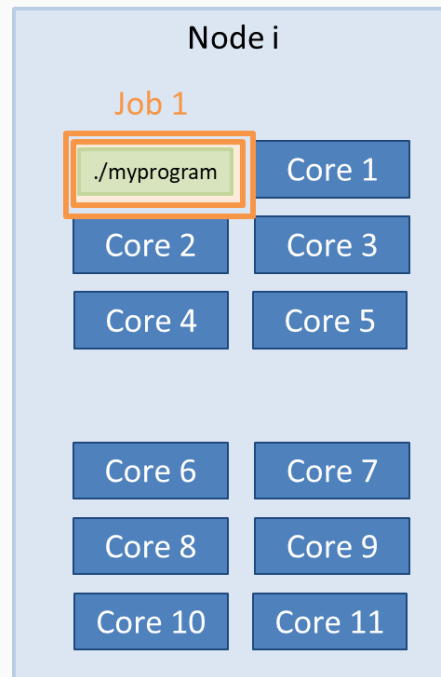
<https://uni-bonn.sciebo.de/apps/forms/s/WJNyYZyBfKF2nsAL2jTrxJby>

1. `sbatch --nodes=2 --ntasks-per-node=16 quiz1.sh`
 - 32 cores
2. `sbatch --ntasks=48 --cpus-per-task=2 quiz2.sh`
 - 96 cores
3. `sbatch --gpus=1 quiz3`
 - 8 cores
 - `sbatch quiz4.sh`
 - 1 core

EXAMPLE JOB SCRIPTS

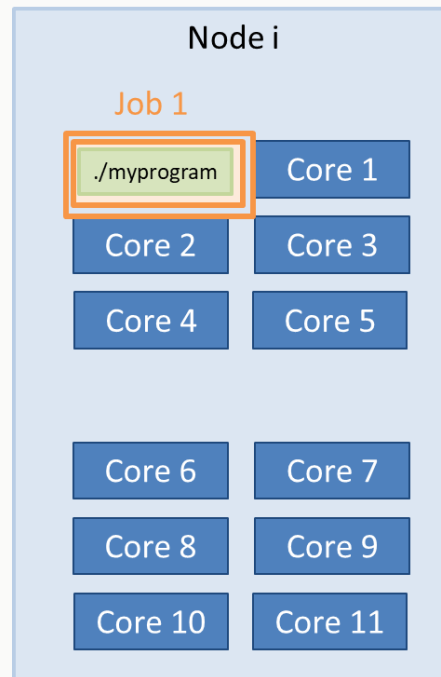
Simple serial program

```
#!/bin/bash  
#SBATCH --time=0:20:00  
#SBATCH --ntasks=1  
#SBATCH --partition=short  
  
./myprogram
```



Simple serial program

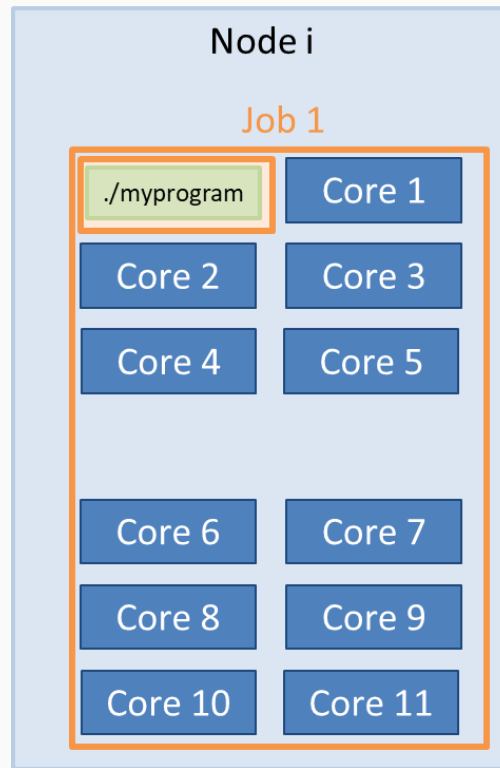
- Simplest possible case
- Number of tasks = number of processes
- Default: only part of RAM used
 - `--mem=X` to select



Wrong: serial program

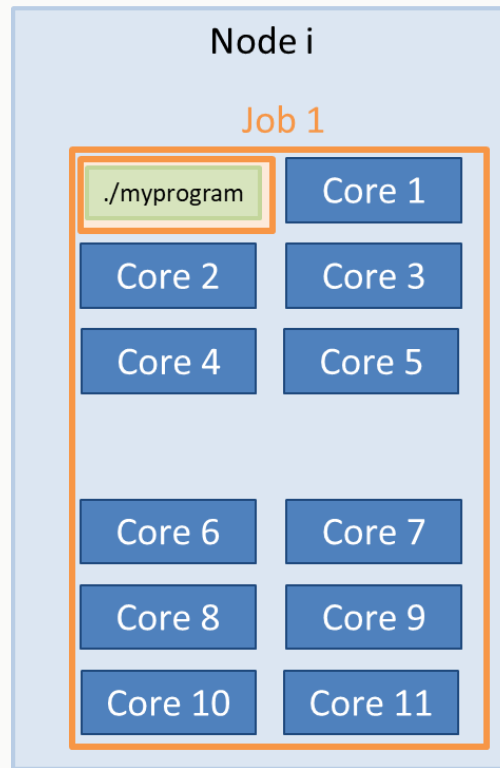
```
#!/bin/bash
#SBATCH --time=0:20:00
#SBATCH --ntasks=12
#SBATCH --partition=short

./myprogram
```



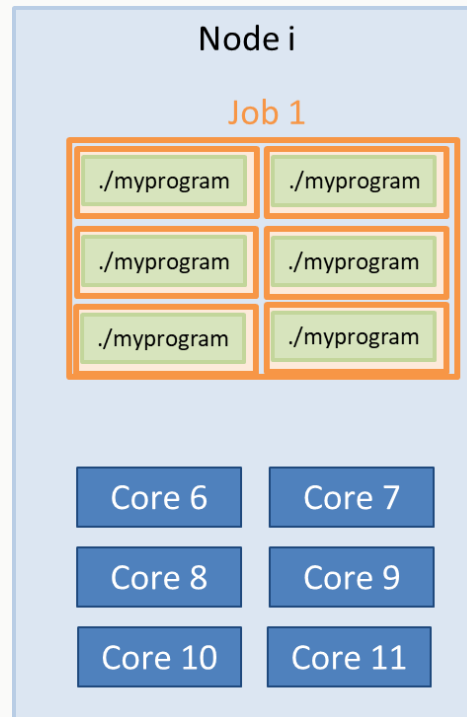
Wrong: serial program

- Waste of resources: full node allocated, only 1 task launched
- Might be legitimate reasons not to use all cores
- But: allocation should reflect reality



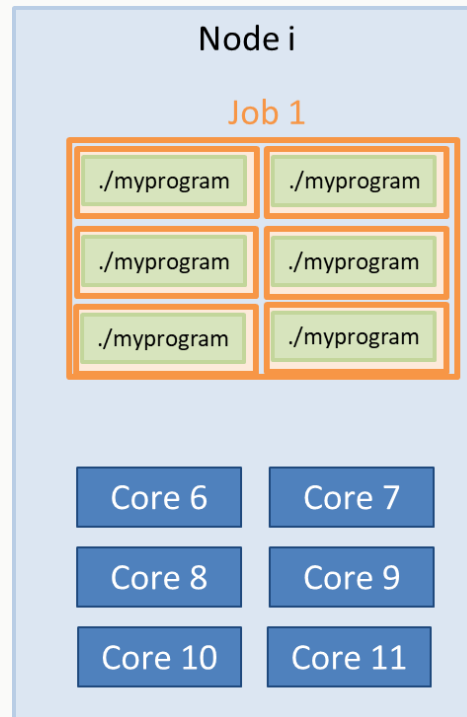
Many instances of serial program

```
#!/bin/bash  
#SBATCH --time=0:20:00  
#SBATCH --ntasks=6  
#SBATCH --partition=short  
  
srun ./myprogram
```



Many instances of serial program

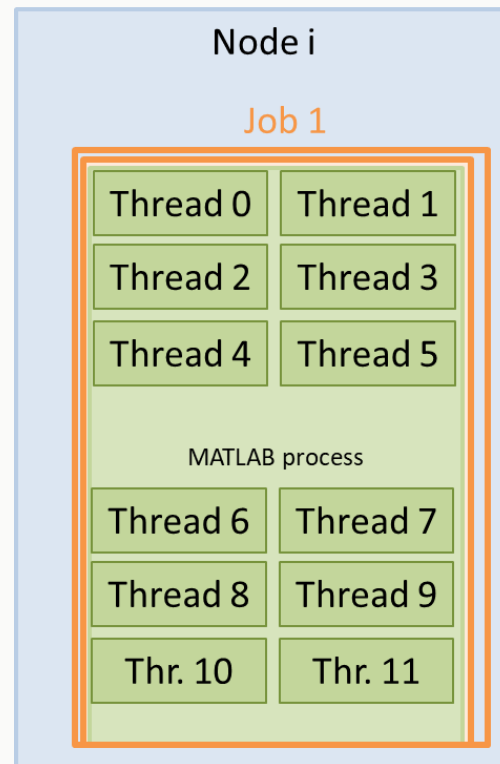
- SLURM simply launches program multiple times
- Cannot talk to each other (without external mechanism)
 - Common PyTorch setup
- Be careful that they write to different files



Multithreaded app (e.g. MATLAB)

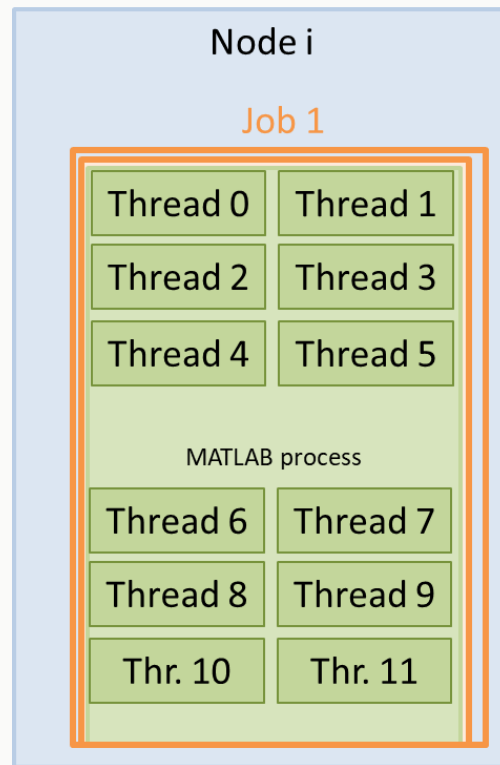
```
#!/bin/bash
#SBATCH --time=0:20:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=12
#SBATCH --partition=short

matlab -nodisplay -r myscript
```



Multithreaded app (e.g. MATLAB)

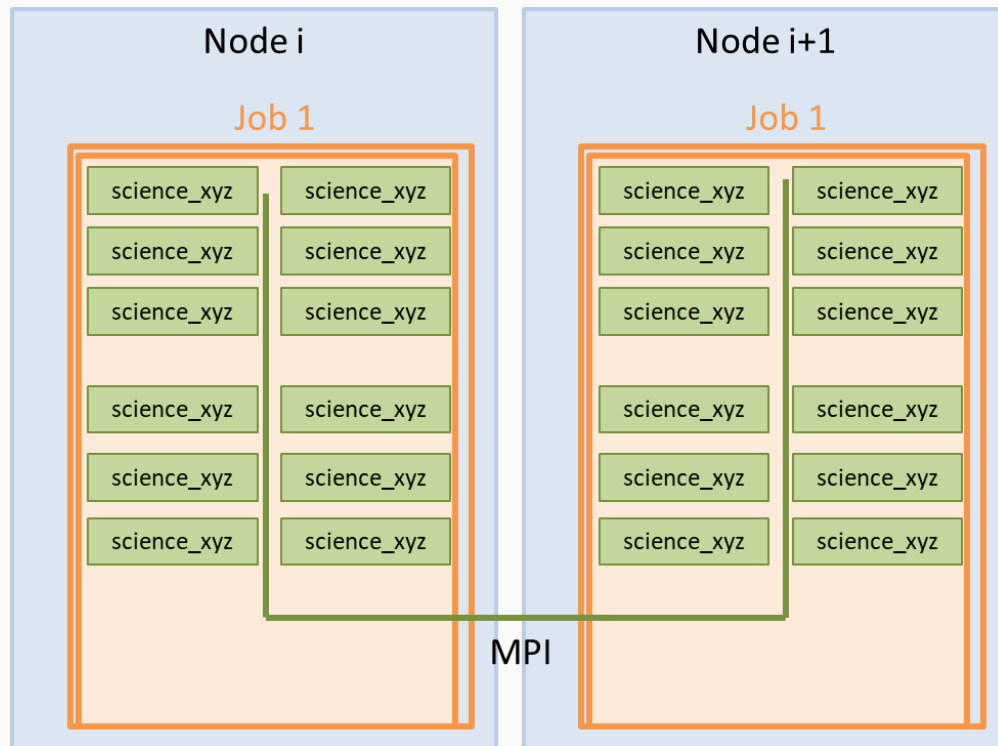
- Common in commercial applications
 - MATLAB: you do not even need to program differently
 - Sometimes good at auto-detecting resources, sometimes not
- Env vars: `SLURM_...`



Big scientific application

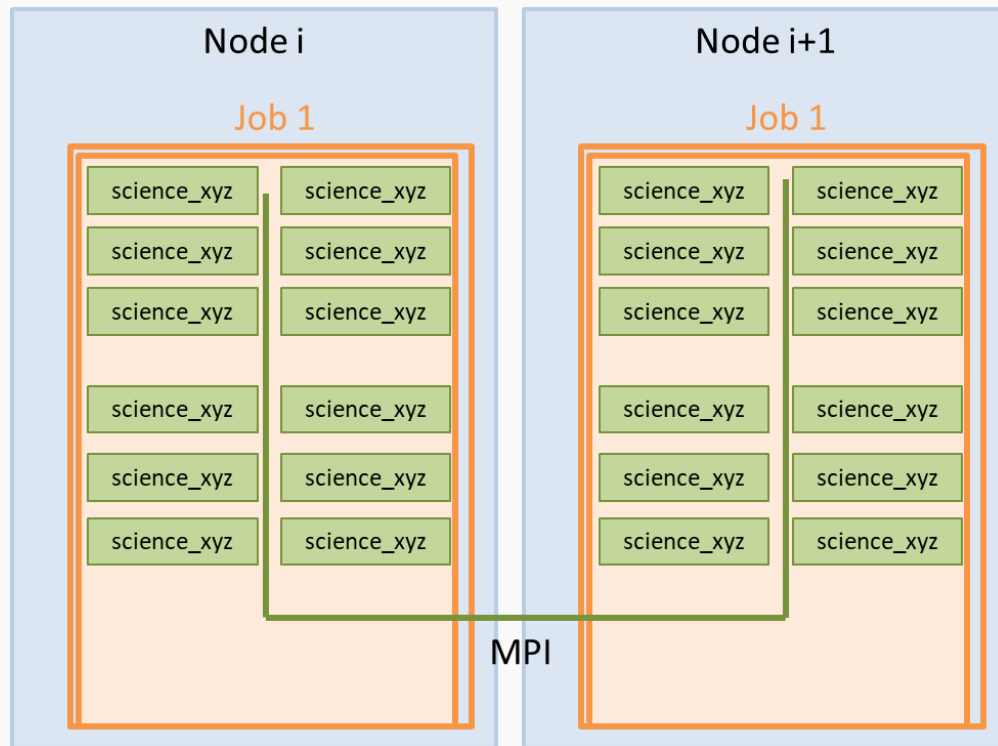
```
#!/bin/bash
#SBATCH --time=4:00:00
#SBATCH --nodes=20
#SBATCH --ntasks-per-node=12

mpirun -np 240 science_xyz
parafire >log.txt
```



Big scientific application

- Many parallel processes
- MPI: common HPC framework
- Launch with `mpirun -np [N]`
- SLURM and MPI can generally talk to each other
- Commercial: sometimes uses internal MPI



- What SLURM settings (tasks and CPUs per task) would you pick for the following cases:
 - You want to run a program with 8 threads
 - You want to run 6 instances of a program (you don't care whether they run on a single node or not)
 - You want to run 64 instances of a program, ensuring that they are distributed evenly over 4 nodes
- Answer the question in the Zoom survey

- What SLURM settings (tasks and CPUs per task) would you pick for the following cases:
 - You want to run a program with 8 threads
 - You want to run 6 instances of a program (you don't care whether they run on a single node or not)
 - You want to run 64 instances of a program, ensuring that they are distributed evenly over 4 nodes



<https://uni-bonn.sciebo.de/apps/forms/s/RR4yKxQdATjHA5o7wKXr5SfY>

- You want to run a program with 8 threads
 - `--ntasks=1 --cpus-per-task=8`
- You want to run 6 instances of a program (you don't care whether they run on a single node or not)
 - `--ntasks=6`
- You want to run 64 instances of a program, ensuring that they are distributed evenly over 4 nodes
 - `--nodes=4 --ntasks-per-node=16`

HOW DOES PYTORCH DO IT?

Based on research and **limited experience**

- PyTorch distributed: multiple backends: `mpi`, `nccl`
- Seems to work with all parallelization strategies (DDP, FSDP, ...)
 - Content of later sections
- Nvidia Collective Communications Library (NCCL): multi-node backend
- Combination of `srun` and "rendezvous" node

Example (not necessarily best or most common way)

Retrieve first node of job

Randomly pick network port

```
MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
MASTER_PORT=$((20000 + $SLURM_JOB_ID % 10000))
```

```
srun -u torchrun
  --nproc_per_node "$SLURM_GPUS_PER_NODE" \
  --nnodes "$SLURM_JOB_NUM_NODES" \
  --rdzv_backend c10d \
  --rdzv_id "$SLURM_JOB_ID" \
  --rdzv_endpoint "${MASTER_ADDR}:${MASTER_PORT}"
  yourscript.py
```



Caution:

Subject to cluster's
firewall settings

Source: <https://gist.github.com/maxidl/844160a41d7a4d6d8bc99af24eb1d208>

Example (not necessarily best or most common way)

One task per node

Shortcut for

```
from torch.distributed.run import main
main()
```

`srun -u torchrun`

```
--nproc_per_node "$SLURM_GPUS_PER_NODE" \
--nnodes "$SLURM_JOB_NUM_NODES" \
--rdzv_backend c10d \
--rdzv_id "$SLURM_JOB_ID" \
--rdzv_endpoint "${MASTER_ADDR}:${MASTER_PORT}" \
yourscript.py
```

One Python process per GPU

PyTorch distributed backend

Unique number

Source: <https://gist.github.com/maxidl/844160a41d7a4d6d8bc99af24eb1d208>

EXERCISE: YOUR FIRST HPC JOB

Objectives

- You can retrieve information about cluster and jobs from SLURM

Tasks

- Find out Bender node and queue setup with `sinfo`
- Compare with HPC website/wiki
- List jobs with `squeue`
- View detailed information with `scontrol show job`, examine differences

Hints

First exercise of 3. Note the following slides!

Objectives

- Run your first job

Tasks

- Write a job script that does the following
 - Request one GPU for 5 minutes
 - Print its own working directory
 - Prints its own job ID
 - Sleeps for 30 seconds
 - Prints "Done"
- Queue it on Bender in the `A40devel` queue

Hints

- Relevant Linux commands, `pwd`, `echo`, `sleep`

Second exercise of 3. Note the following slide!

Objectives

- If bored, get creative

Tasks

- Try out different SLURM parameters
- Try a parallel PyTorch "Hello World"
- ...

Third exercise of 3.

DYNAMIC SECTIONS

ENVIRONMENT MODULES

- What happens in Linux if you type a command?
 - `PATH` environment variable: list of directories
 - Set by Linux
 - Modified by installed software
 - Modifiable by you
 - Other system settings: similar principle
- Environment variables: way of setting system configuration

- Cluster: conflicting needs
 - E.g.: `matlab` command:
 - User A needs MATLAB 2023
 - User B need MATLAB 2024
- HPC clusters: modular environment
- You have control which software

- Users load (and unload) modules
- We: provide software in module form
- Also: dependency management

- What software is installed?
 - `module avail` → Overview available modules
 - `module spider` → Search function
- What does a module do?
 - `module help` → Help text
 - `module show` → Environment changes

- Handle modules

- `module load/module unload`
- `module purge` → unload everything (debugging)

- What is currently loaded?

- `module list`
- Marked (L) in `module avail` output

```
$ module avail
```

- Module hierarchy similar to folders
- Many variants
- One marked as default: (D)

```
GCC/11.2.0  
GCC/11.3.0  
GCC/12.2.0  
GCC/12.3.0  
GCC/13.2.0  
GCC/13.3.0 (D)
```

- `module load GCC` → Default version `GCC/13.3.0` loaded
- `module load GCC/11.2.0` → pick version

- Software: LMod
 - Worth reading documentation
 - https://lmod.readthedocs.io/en/latest/010_user.html
- Cool and many advanced features
 - Create own modules
 - Define sets of modules

Match the module commands with their functions

- | | |
|-----------------------------|------------------------------------|
| - <code>module show</code> | - Show what modules exist |
| - <code>module list</code> | - Show what modules are loaded |
| - <code>module avail</code> | - Load a module |
| - <code>module help</code> | - Unload a module |
| - <code>module load</code> | - Show help text |
| | - Show what changes a module makes |

Match the module commands with their functions

- module show
- module list
- module avail
- module help
- module load



<https://uni-bonn.sciebo.de/apps/forms/s/J4NZm3pmiEcNPbTNpFiQjPkb>

Match the module commands with their functions

- `module show` → Show what changes a module makes
- `module list` → Show what modules are loaded
- `module avail` → Show what modules exist
- `module help` → Show help text
- `module load` → Load a module
- No match: Unload a module

FILE SYSTEMS AND WORKSPACES

- HPC: large amounts of data
 - Specialized filesystems
 - Marvin: several problems with filesystems overflowing
- Responsible cluster use == familiarize yourself with file systems

- Your home directory
 - Code
 - Hard limit: 100 GB
- Marvin: workspaces
 - Unlimited in size (5 PB capacity)
 - But: limited in time (4 x 90 days)

- Marvin workspaces: managed with commands
 - `WS_...`
- HPC-Workspaces software
 - <https://wiki.hpc.uni-bonn.de/marvin/workspaces>
 - <https://github.com/holgerBerger/hpc-workspace/blob/master/user-guide.md>

- `ws_allocate <name> <days>`, `ws_extend <name> <days>`
 - Create new workspace, extend existing WS
 - Caution: always specify days (max 90) → data deleted after expiration (rescueable for 21 days)
- `ws_list` → List your workspaces
- `ws_release <name>` → Free up workspace

- Configure e-mail notifications

- `ws_allocate -r 3 -m my_addr@uni-bonn.de my_ws_name 30`

- `-r X` → Reminder X days before expiration

- User guide: create simple user config file → saves you from typing address every time

- `~/ .ws_user.conf`

- [See user guide](#)

INTERACTIVE JOBS

Scenario:

- You want to use a CPU-intensive application yourself
 - e.g. visualization, post-processing
- Will slow down/block entire login node
 - We reserve the right to kill processes
- You cannot start a batch job because application needs your input

Solution:

- Interactive job
 - Resources allocated like any other job
 - Same SLURM parameters
 - But only a console is opened and you can work within it
 - Parameter `--pty`: makes it interactive

```
$ srun --pty --time 5:00:00 /bin/bash
```

THANK YOU FOR YOUR ATTENTION

HPC@HRZ

J. Steiner, F. Boecker, K. Ferchichi

training@hpc.uni-bonn.de